

Tutorial 05

Keys

- The **super key** of a schema is a **set** of attributes that **uniquely identifies** each tuple in a relation **for any instance**.
- The **candidate key** is a **minimal** (not minimum) super key.
- The **primary key** is a candidate key that is the **most appropriate** to become the main key of the relation, never or rarely changed.

Table: student

id	name	year	gpa	phone	major
123	Richard	2	3.9	1342225648	CST
234	Grace	3	3.4	1526482319	DS
345	Bob	3	3.2	1865482164	AI

Super key: $\{id, gpa\}$, $\{phone, gender\}$, ...

Candidate key: $\{id\}$, $\{phone\}$

Primary key: $\{id\}$

- $\{year, gpa\}$ **cannot** be a super key even it can uniquely identify the tuples in this example. It cannot identify in some possible instances.

Foreign Keys

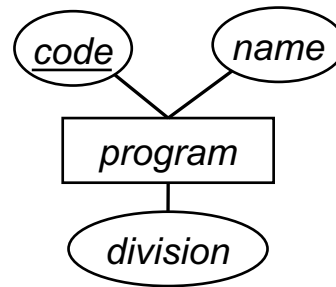
- A relational schema may have a set of attribute(s) that corresponds to the primary key of another relation. This set of attribute(s) is called a **foreign key**.
 - For example, given the following schemas
$$\begin{aligned} student &= (\underline{student_ID}, name, major, phone, year, GPA) \\ book &= (\underline{ISBN}, title, author) \\ borrow &= (\underline{student_ID}, \underline{ISBN}) \end{aligned}$$
 - The attribute *ISBN* in the schema *borrow* is a foreign key to the schema *book*, same for *student_ID*.
 - *book* is the **referenced relation**. And *borrow* is the **referencing relation**.
 - The purpose of a foreign key is to find some information across multiple tables.
 - Only values occurring in the primary key attribute of the **referenced relation** may occur in the foreign key attribute of the **referencing relation**.

Foreign Keys

- The purpose of a foreign key is to find some information **across multiple tables.**
- For example, to find the names of books borrowed by David,
 - one must search David's student ID in the table *borrow*;
 - then, he knows the ISBNs of books borrowed by David;
 - next, he can search for the book names using the ISBNs from the table *book*.
- To make sure the searching can always be done, only values occurring in the primary key attribute of the referenced relation may occur in the foreign key attribute of the referencing relation.
- For example, it's impossible to let *borrow* have an ISBN which does not exist in *book*.

Entity Sets

- Converting strong entity sets (without composite attributes or multivalued attributes) is straightforward.
- For example, the entity set “program”



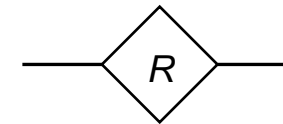
is represented by $program = (\underline{code}, name, division)$.

- For weak entity sets, the case is same as many-to-one relationship set with total participation on the many side (case 9 on page 16).

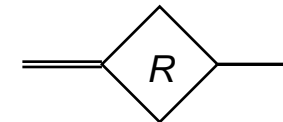
Relationship Sets

- Different constraints make the conversion of relationship sets different.
- Here we list all different cases of the constraints. (10 in total)

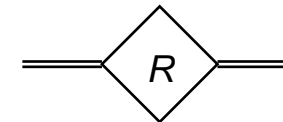
1. many-to-many without total participation



2. many-to-many with total participation on one side

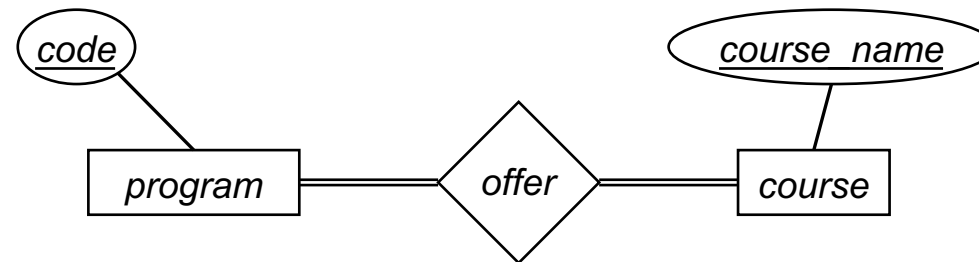


3. many-to-many with total participation on both sides



Relationship Sets

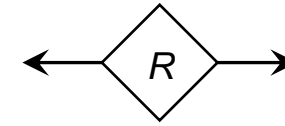
- For the above 3 cases, we create an individual schema for each relationship set.
- The column names consist of the key attributes for both entity sets and the attributes for the relationship set (if any).
- The key of the schema is the combination of the key attributes for both entity sets.
- For example, the relationship set “offer” is presented as
$$offer = (\underline{code}, \underline{course_name})$$



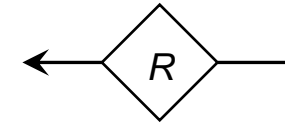
Relationship Sets

- More cases:

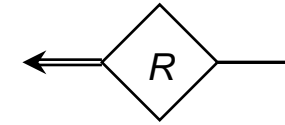
4. one-to-one without total participation



5. one-to-many without total participation



6. one-to-many with total participation on the “one” side

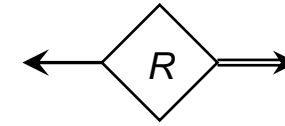


- For these three cases, the conversion is same as the previous cases.
- But the key of the schema is
 - either one of the key attributes of the entity sets (case 4);
 - the key attribute of the entity set on the “many” side (case 5 and 6).

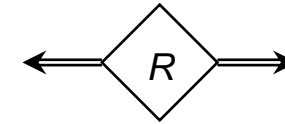
Relationship Sets

- More cases:

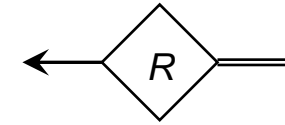
7. one-to-one with one total participation and one partial participation



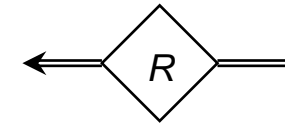
8. one-to-one with total participation on both sides



9. one-to-many with total participation on the “many” side

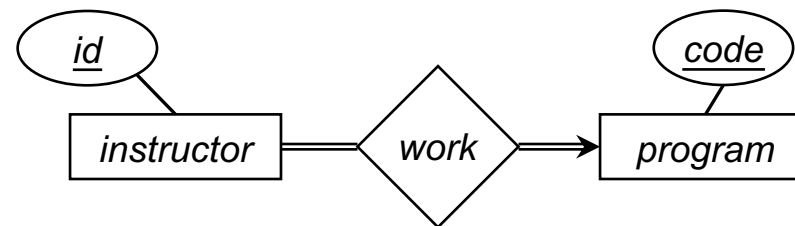


10. one-to-many with total participation on both sides



Relationship Sets

- For these four cases, there is no need to create a schema.
- We only need to add a foreign key as a reference to the schema of
 - the entity set with total participation on the “one” side (case 7 and 8)
 - the entity set with total participation on the “many” side (case 9 and 10)
- For example, the relationship set “work” associating “instructor” and “program”.

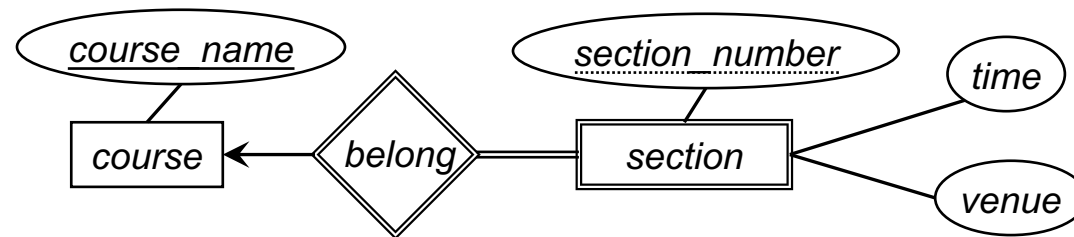


foreign key

- Because every instructor is associated with exactly one program, there is no need to create a schema for “work”.
- The schema for “instructor” is $instructor = (\underline{id}, \dots, \boxed{code})$.

Relationship Sets

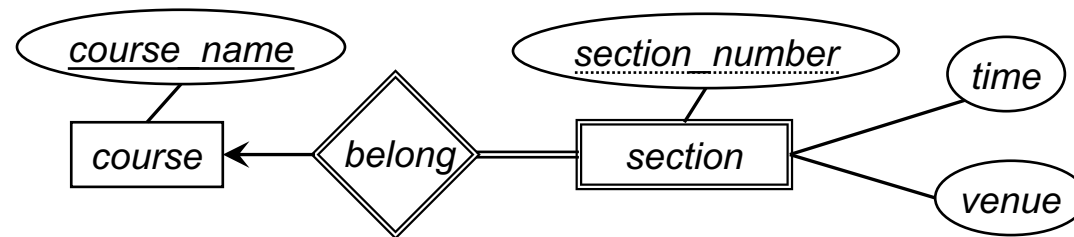
- The conversion for weak entity sets is same as case 9, one-to-many with total participation on the “many” side.
- The only difference is the key.
- For example,



- The schema for the entity set “section” is
 $section = (\underline{course_name}, \text{section_num}, \text{time}, \text{venue})$
- There is no need to create a schema for “belong”.

Relationship Sets

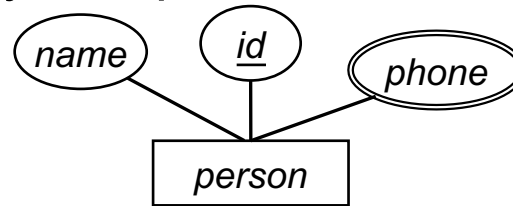
- The conversion for weak entity sets is same as case 9, one-to-many with total participation on the “many” side.
- The only difference is the key.
- For example,



- The schema for the entity set “section” is
 $section = (\underline{course_name}, \underline{section_num}, time, venue)$
- There is no need to create a schema for “belong”.

Multivalued Attributes

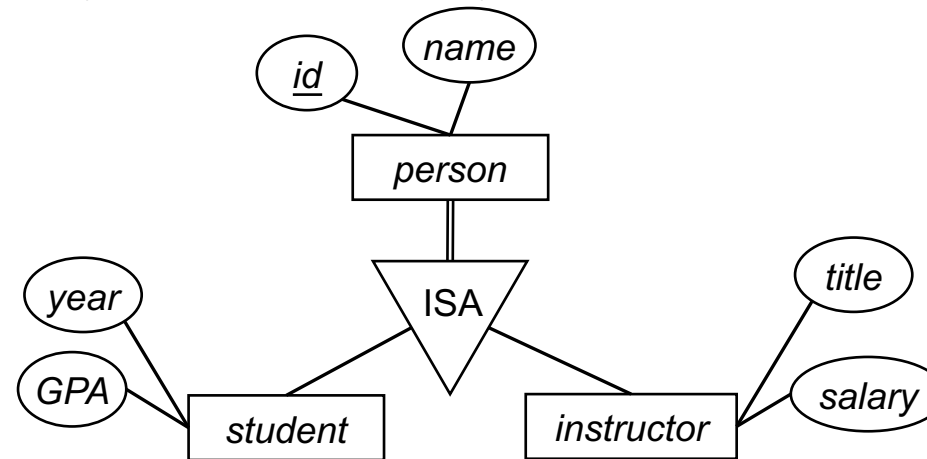
- If an entity set has a multivalued attribute, the entity set is converted into two relational schemas.
- One consists of all attributes except the multivalued attribute.
- The other one consists of the key attribute(s) and the multivalued attribute. The key of this schema is the schema itself.
- For example, the entity set “person”



is converted into $person = (id, name)$ and $person_phone = (id, phone)$.

ISA Relationship Sets

- If the ISA relationship set is overlapping or partial
 - the schema for the higher-level entity set is like other strong entity sets;
 - the schema for the lower-level entity set only consists of the attributes of this lower-level entity set and the **key** of the higher-level entity set;
 - the key of the higher-level entity set is also the key of the lower-level entity set.
- Assume that the multivalued attribute “phone” is omitted, and the ISA is not disjoint.



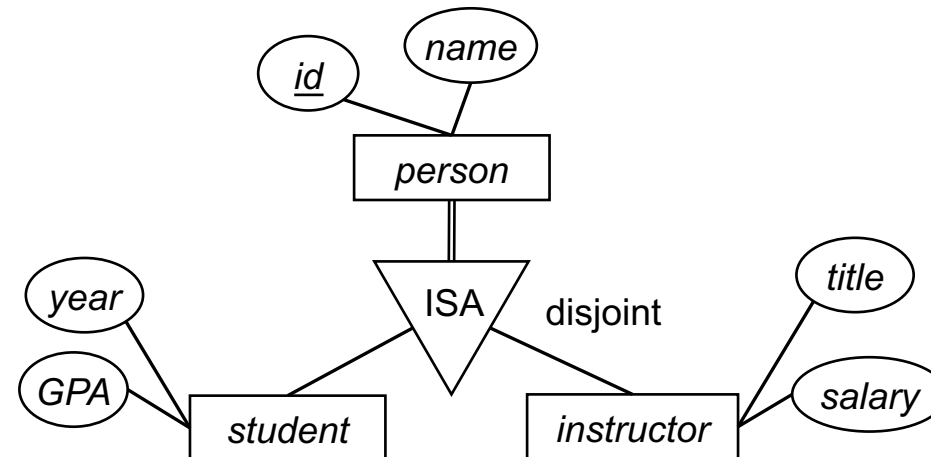
$person = (\underline{id}, name)$

$student = (\underline{id}, year, GPA)$

$instructor = (\underline{id}, title, salary)$

ISA Relationship Sets

- If the ISA relationship set is disjoint **and** total
 - no need to create a schema for the entity set on the higher level;
 - one schema is constructed for each entity set on the lower level, inheriting all attribute from the higher-level entity set.



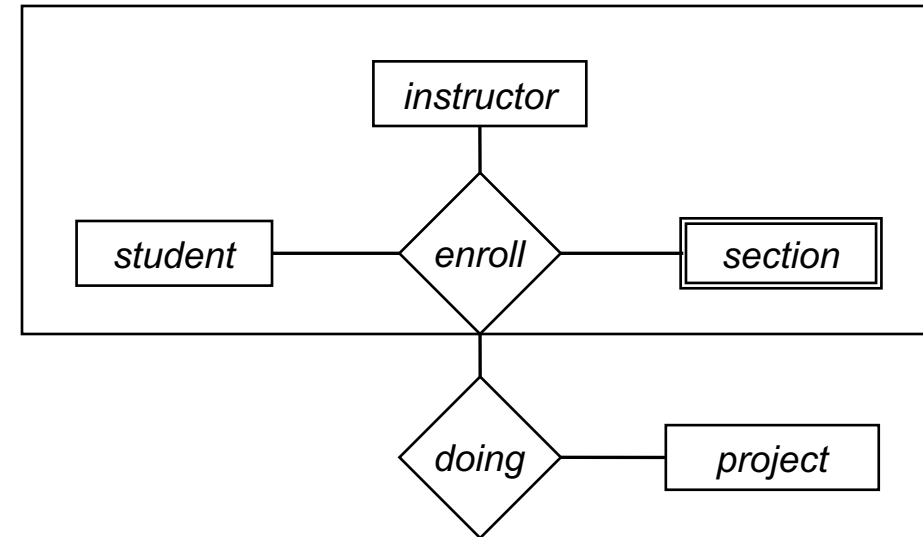
$student = (\underline{id}, name, year, GPA)$

$instructor = (\underline{id}, name, title, salary)$

Aggregation

- Aggregations are treated same as other relationship sets.

- For example, the schema for “enroll” has the key of “instructor”, “student”, and “section”.
- Then, “doing” is a binary many-to-many relationship set. And the method for case 1 can be applied.
- Thus,

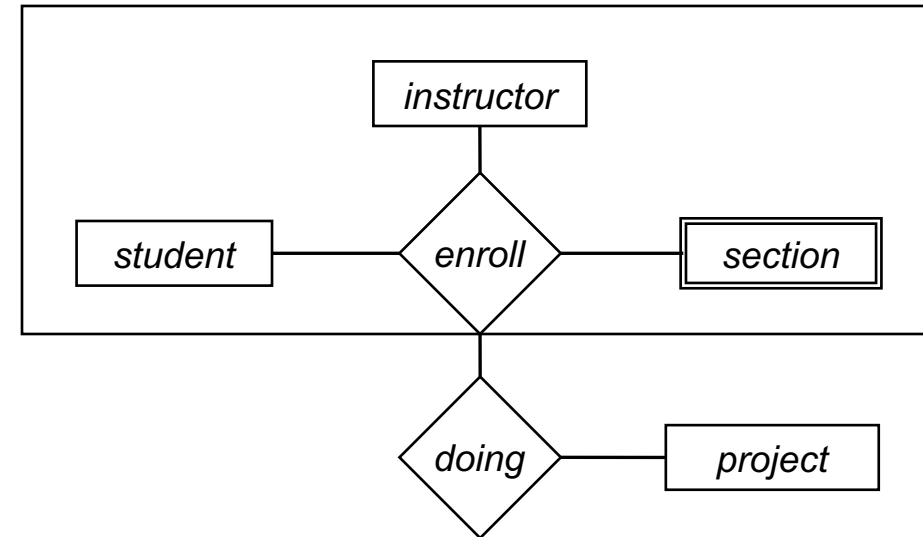


$doing = (\underline{student_ID, instructor_ID, course_name, section_number, semester, project_ID})$

Aggregation

- Aggregations are treated same as other relationship sets.

- For example, the schema for “enroll” has the key of “instructor”, “student”, and “section”.
- Then, “doing” is a binary many-to-many relationship set. And the method for case 1 can be applied.
- Thus,



$doing = (\underline{student_ID, instructor_ID, course_name, section_number, semester, project_ID})$

Lecture6 exercise

REVIEW

programs = (p_code, p_name, division, director)

course = (c_name, credits, domain, c_number)

offer = (p_code, c_name)

section =
(c_name, s_number, sem, venue, time, instructor_id)

enroll = (id, c_name, s_number, sem, grade)

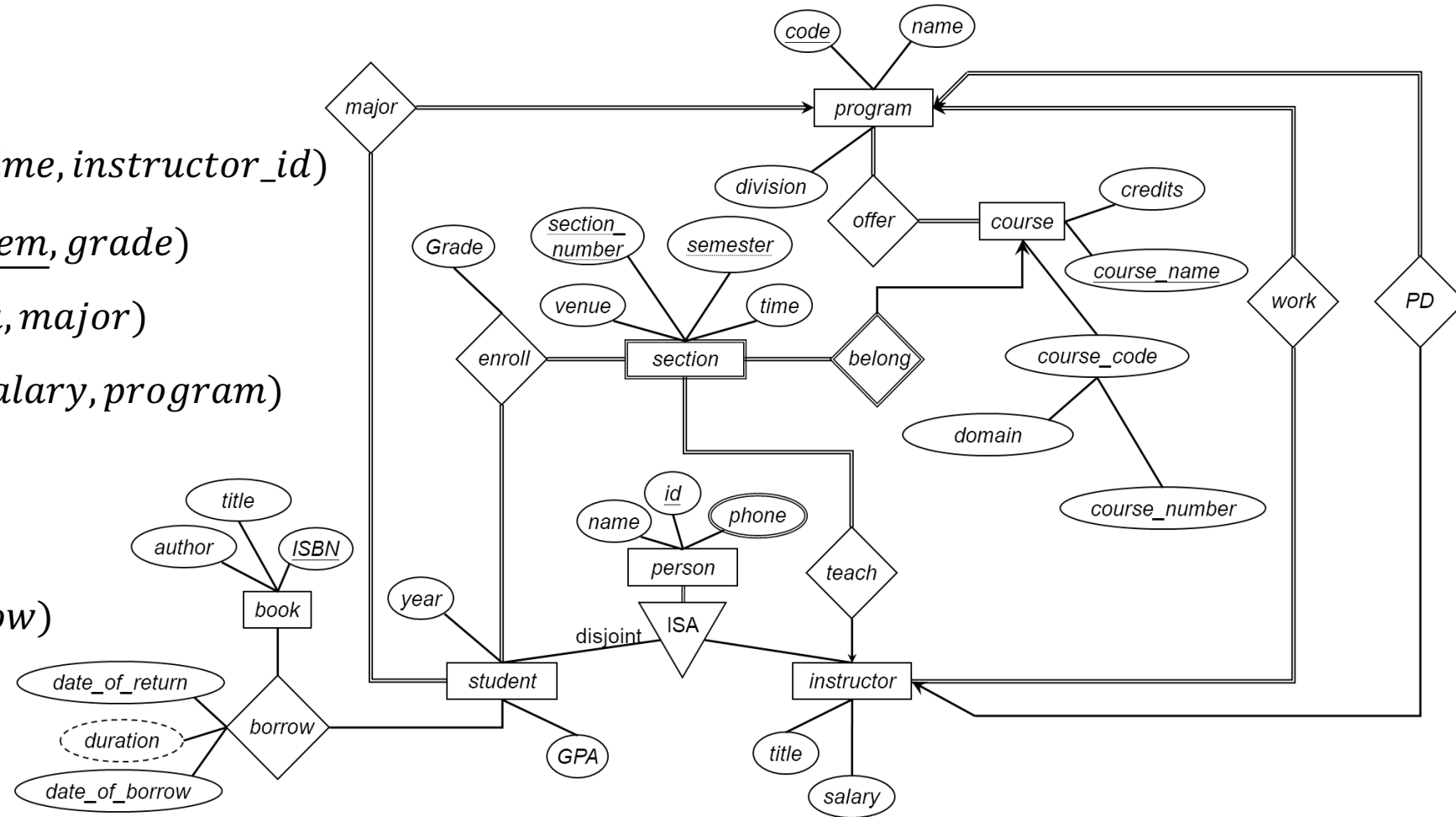
students = (id, s_name, year, gpa, major)

instructors = (id, i_name, title, salary, program)

contact = (id, phone)

books = (ISBN, b_title, author)

borrow = (id, ISBN, *return*, borrow)



programs = (p_code, p_name, division, director)

course = (c_name, credits, domain, c_number)

offer = (p_code, c_name)

section =

(c_name, s_number, sem, venue, time, instructor_id)

enroll = (id, c_name, s_number, sem, grade)

students = (id, s_name, year, gpa, major)

instructors = (id, i_name, title, salary, program)

contact = (id, phone)

books = (ISBN, b_title, author)

*borrow = (id, ISBN, **return**, borrow)*

Lab.6

REVIEW

Create Tables

To create tables, we use

```
CREATE TABLE table_name (  
    attribute1 type1,  
    attribute2 type2,  
    ...  
    constraint1,  
    constraint2,  
    ...  
)
```

“table_name” is the name of the table.

“attribute” is the name of the attribute.

“type” is the data type for the attribute.

“constraint” is a condition on the table. If users try to insert some data violating the condition, the system will give a warning.

Data Types

char(<i>n</i>)	fixed length character string of at most length <i>n</i> .
varchar(<i>n</i>)	variable length character string of at most length <i>n</i> .
int	integer (a finite subset of the integers that is machine-dependent).
smallint	small integer (a machine-dependent subset of the integer domain type).
numeric(<i>p</i>,<i>n</i>)	fixed point number, with user-specified precision of <i>p</i> digits, and with <i>n</i> digits to the right of decimal point.
real	floating point, with machine-dependent precision.
float(<i>n</i>)	floating point number, with user-specified precision of at least <i>n</i> digits.
year	4-digit string or 4-digit number from 1901 to 2155.
date	in 'YYYY-MM-DD' format from 1000-01-01 to 9999-12-31.
time	in 'HH:MM:SS' format.
blob	binary large object, usually for images.
clob	character large object, usually for long text.

Foreign key

books = (ISBN, b_title, author)

borrow = (id, ISBN, return_date, borrow_date)

A value of “ISBN” in “borrow” cannot exist if such an ISBN is not in “books”. (How do you borrow a book which does not exist?)

Thus, “ISBN” in “borrow” is a foreign key referencing to “ISBN” in “books”.

When users try to borrow a book (insert a row to table “borrow”), the system should check the existence.

FOREIGN KEY (attribute1) REFERENCES table2(attribute2)

- “attribute1” in the current table and “attribute2” in “table2” represent the same piece of information.
- “attribute2” is the primary key for “table2”.
- A value of “attribute1” in the current table **cannot exist alone**.
- More will be on this in the following labs.

Foreign key

To add foreign keys to an existing table, we need to change the schema.

```
ALTER TABLE borrow ADD FOREIGN KEY (ISBN) REFERENCES books(ISBN)
```

ALTER TABLE: the keyword to change the schema of a table.

borrow: the table name.

ADD: to add new attributes or constraints (“DROP” to remove attributes or constraints).

FOREIGN KEY: the constraint.

(ISBN): the foreign key.

REFERENCES: the keyword to indicate which attribute is the target.

books (ISBN): the attribute “ISBN” in the table “books” is referenced.

Exercises

Implement the logical design in MySQL

Create one table for each of the schema.

Define the primary key.

Define the suitable data type for each attribute.

Define the foreign keys.

Exercises ans

See lab 7 ->uic definition.txt